

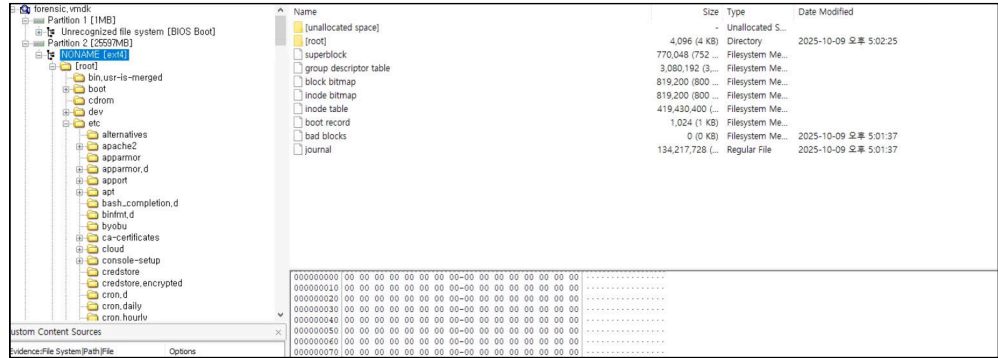
문제 풀이

문제 유형
/ 출제 분야

[시나리오] 시나리오 1-3

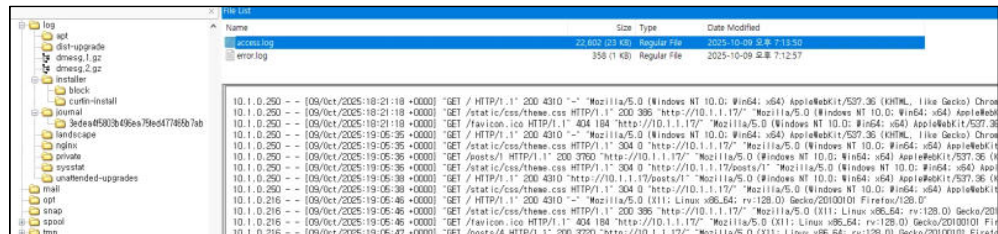
문제 요약
(설명)

주어진 디스크 이미지 내에서 백도어가 설치된 경로를 찾아내어, 제출
(이전 시나리오 내용 및 관련 웹 로그 흔적 활용)



문제
4

1-3 문제에서 주어진 디스크 이미지를 FTK Imager를 통해 확인한 결과, 리눅스 디스크 이미지 파일이라는 점을 확인할 수 있었으며, 기존 1-1 문제를 구동한 서버 이미지 파일이라는 것을 확인할 수 있었습니다.

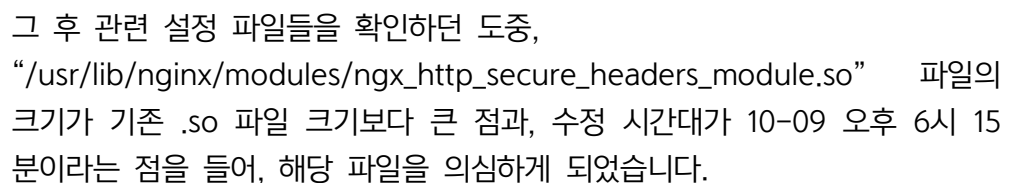


시나리오에 따라, 기존 1-1/1-2의 풀이 내용을 통해 확인한 결과, PyYAML 파싱시의 취약점(CVE-2020-14343)을 통해 최초 침투 이후, 백도어 설치 및 접근을 진행했을 것으로 파악되었습니다.

그렇기에 먼저 관련 웹 로그를 먼저 확인해야 한다고 판단하여, log/nginx/access.log 파일을 먼저 확인하였습니다.

```
- 10.1.0.121 - - [09/Oct/2025:19:12:57 +0000]
"GET /bd HTTP/1.1" 200 5 "-" "curl/7.81.0 "
```

해당 분석을 통해서 Nginx와 관련된 설정 파일 및 바이너리와 관련하여 문제가 있을 것이라 예상하였으며, 관련된 파일들을 확인하기 시작했습니다.



Module ngx_http_secure_link_module

[Directives](#)

[secure_link](#)

[secure_link_md5](#)

[secure_link_secret](#)

[Embedded Variables](#)

The `ngx_http_secure_link_module` module (0.7.18) is used to check authenticity of requested links, protect resources from unauthorized access, and limit link lifetime.

The authenticity of a requested link is verified by comparing the checksum value passed in a request with the value computed for the request. If a link has a limited lifetime and the time has expired, the link is considered outdated. The status of these checks is made available in the `$secure_link` variable.

The module provides two alternative operation modes. The first mode is enabled by the [secure_link_secret](#) directive and is used to check authenticity of requested links as well as protect resources from unauthorized access. The second mode (0.8.50) is enabled by the [secure_link](#) and [secure_link_md5](#) directives and is also used to limit lifetime of links.

This module is not built by default, it should be enabled with the `--with-http_secure_link_module` configuration parameter.

그 후, 추가적인 확인 절차를 통해 ngx_http_secure_link_module에 대해 조사한 결과, 기존 모듈의 동작이 보안과 관련된 여러 HTTP(Header)들을 자동으로 추가해주는 점과, default로 빌드되지 않는다는 점을 들어, 해당 파일이 공격자가 설치한 백도어 파일이라는 것을 확신하게 되었습니다.

이후, 위 조사 결과에 따라, 해당 파일 경로를 플래그로 제출하게 되었습니다.

최종 플래그는 아래와 같습니다.

```
whitehat2025{/usr/lib/nginx/modules/ngx_http_secure_headers_module  
.so}
```

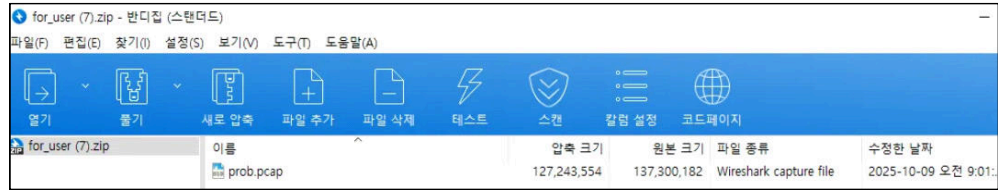
문제 풀이

문제 유형
/ 출제 분야

[포렌식] Leakage Investigation

문제 요약
(설명)

주어진 패킷 파일을 zeek 도구를 활용한 분석 후, 의심 IP를 찾은 뒤, WireShark를 활용해 유출 활용 파일(스태가노그래피) 및 대화 내용 분석



해당 문제의 경우, 하나의 pcap 파일이 제공되는 문제였습니다.

해당 pcap 파일을 분석하기 위해, zeek를 활용하여 분석을 진행하였습니다.

문제
7

```

zeek_lab cat weird.log
#separator \x09
#set_separator ,
#empty_field (empty)
#unset_field
#path weird
#open 2025-10-18-13-55-45
#fields ts uid id.orig_h id.orig_p id.resp_h id.resp_p name addl notice peer source
#types time string addr port addr port string string bool string string
1758684924.046171 - - - - 192.168.110.128 22114 198.51.100.23 80 zeek IP
1758685042.598348 CSK1Aq4J4SfJFzWt4a 192.168.110.128 22114 198.51.100.23 80 bad_HTTP_request -F zeek HTTP
1758685042.598348 CSK1Aq4J4SfJFzWt4a 192.168.110.128 22114 198.51.100.23 80 line_terminated_with_single_CR - F zeek CONTENTLINE
1758685063.708348 CZUp0ZPGH9vF35p7 192.168.110.128 22033 198.51.100.23 80 line_terminated_with_single_CR - F zeek CONTENTLINE
1758685064.818349 CYDAI53KuRmP5KHDe 192.168.110.128 49192 198.51.100.23 80 bad_HTTP_request -F zeek HTTP
1758685064.818349 CYDAI53KuRmP5KHDe 192.168.110.128 49192 198.51.100.23 80 line_terminated_with_single_CR - F zeek CONTENTLINE
1758685075.928349 Cfuvib2yAdOX3MkZ5 192.168.110.128 37695 198.51.100.23 80 line_terminated_with_single_CR - F zeek CONTENTLINE
1758685087.038349 CLMIdr19Fa9C1de805 192.168.110.128 39750 198.51.100.23 80 bad_HTTP_request -F zeek HTTP
1758685087.038349 CLMIdr19Fa9C1de805 192.168.110.128 39750 198.51.100.23 80 line_terminated_with_single_CR - F zeek CONTENTLINE
1758685097.148350 CuzYYGee8LHWIE57 192.168.110.128 43357 198.51.100.23 80 line_terminated_with_single_CR - F zeek CONTENTLINE
1758685108.258350 Qhenc12aQhnmZamfni 192.168.110.128 28055 198.51.100.23 80 bad_HTTP_request -F zeek HTTP
1758685108.258350 Qhenc12aQhnmZamfni 192.168.110.128 28055 198.51.100.23 80 line_terminated_with_single_CR - F zeek CONTENTLINE
1758685119.368351 CZho425uY5He4qze2 192.168.110.128 47829 198.51.100.23 80 line_terminated_with_single_CR - F zeek CONTENTLINE
#close 2025-10-18-13-55-45
zeek_lab

```

zeek를 통해 기본적인 분석을 수행한 이후,

확인된 weird.log에서 bad_HTTP_request(잘못된 HTTP 요청) 및

line_terminated_with_single_CR(라인 (줄)이 단일 CR로 종료됨)과

같은 내용들을 보게 되었으며, 특히 198.51.100.23 IP 주소의 대역대는 문서 및 예제용으로 예약된 특수 IP 대역에 속하는 상태였습니다.

그렇기에 해당 IP는 통상적으로 실제 인터넷상에서 특정 서버나 개인에게 할당되어 사용되는 주소가 아니기에, 해당 IP를 통해 주고 받은 패킷들이 유출에 활용된 패킷들이라고 판단하게 되었습니다.

No.	Time	Source	Destination	Protocol	Length	Info
113482	262.684795	192.168.118.128	198.51.100.23	TCP	54	47829 → 80 [SYN] Seq=0 Win=8192 Len=0
113483	262.684795	198.51.100.23	192.168.118.128	TCP	54	80 → 47829 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0
113484	262.684795	192.168.118.128	198.51.100.23	TCP	54	47829 → 80 [ACK] Seq=1 Ack=1 Win=8192 Len=0
113485	262.924795	192.168.118.128	198.51.100.23	TCP	54	[TCP Keep-Alive] 47829 → 80 [ACK] Seq=0 Ack=1000 Win=8192 Len=0
113486	262.934795	192.168.118.128	198.51.100.23	TCP	54	[TCP Keep-Alive] 47829 → 80 [ACK] Seq=0 Ack=2400 Win=8192 Len=0
113489	262.948795	192.168.118.128	198.51.100.23	TCP	54	[TCP Keep-Alive] 47829 → 80 [ACK] Seq=0 Ack=3000 Win=8192 Len=0
113492	262.968795	192.168.118.128	198.51.100.23	TCP	54	[TCP Keep-Alive] 47829 → 80 [ACK] Seq=0 Ack=4000 Win=8192 Len=0
113504	262.972795	192.168.118.128	198.51.100.23	TCP	54	[TCP Keep-Alive] 47829 → 80 [ACK] Seq=0 Ack=5500 Win=8192 Len=0
113505	262.974795	198.51.100.23	192.168.118.128	TCP	54	80 → 47829 [FIN, ACK] Seq=5500 Ack=0 Win=8192 Len=0
113506	262.984795	192.168.118.128	198.51.100.23	TCP	54	[TCP Keep-Alive] 47829 → 80 [ACK] Seq=0 Ack=5597 Win=8192 Len=0
113764	214.984795	198.51.100.23	192.168.118.128	TCP	54	60000 → 54321 [FIN, ACK] Seq=164 Ack=556 Win=8192 Len=0
113765	214.984795	192.168.118.128	198.51.100.23	TCP	54	54321 → 60000 [ACK] Seq=156 Ack=105 Win=8192 Len=0
33789	112.114782	192.168.118.128	198.51.100.23	TCP	100	54321 → 60000 [PSH, ACK] Seq=138 Ack=164 Win=8192 Len=46
25288	184.114782	198.51.100.23	192.168.118.128	TCP	101	60000 → 54321 [PSH, ACK] Seq=117 Ack=138 Win=8192 Len=47
17758	67.114782	198.51.100.23	192.168.118.128	TCP	106	60000 → 54321 [PSH, ACK] Seq=0 Ack=1 Win=8192 Len=52
22824	93.114782	192.168.118.128	198.51.100.23	TCP	100	54321 → 60000 [PSH, ACK] Seq=56 Ack=117 Win=8192 Len=54
20866	93.114782	198.51.100.23	192.168.118.128	TCP	119	60000 → 54321 [PSH, ACK] Seq=52 Ack=56 Win=8192 Len=65
183908	188.742784	198.51.100.23	192.168.118.128	TCP	721	80 → 43357 [PSH, ACK] Seq=4800 Ack=0 Win=8192 Len=667
84881	137.382783	198.51.100.23	192.168.118.128	TCP	777	80 → 22853 [PSH, ACK] Seq=4800 Ack=0 Win=8192 Len=723
86556	155.522783	198.51.100.23	192.168.118.128	TCP	784	80 → 37895 [PSH, ACK] Seq=4800 Ack=0 Win=8192 Len=738
185520	131.852785	192.168.118.128	198.51.100.23	TCP	823	28955 → 80 [PSH, ACK] Seq=4800 Ack=0 Win=8192 Len=769
113493	262.982795	198.51.100.23	192.168.118.128	TCP	858	80 → 47829 [PSH, ACK] Seq=4800 Ack=0 Win=8192 Len=796
91081	146.412783	192.168.118.128	198.51.100.23	TCP	852	49312 → 80 [PSH, ACK] Seq=4800 Ack=0 Win=8192 Len=798
56623	126.192782	192.168.118.128	198.51.100.23	TCP	878	22114 → 80 [PSH, ACK] Seq=4800 Ack=0 Win=8192 Len=816

Frame 33789: Packet, 100 bytes on wire (800 bits), 100 bytes captured (800 bits) on interface 0

Ethernet II, Src: VMware_vlans15 (00:0c:29:a1:1e:15), Dst: VM-NB-PhysServer-16 (00:10:20:00:00:23)

Internet Protocol Version 4, Src: 192.168.118.128, Dst: 198.51.100.23

Transmission Control Protocol, Src Port: 54321, Dst Port: 60000, Seq: 117, Ack: 164, Len: 46

Data (46 bytes)

그 후, wireshark 내에서 (ip.addr == 198.51.100.23)으로 필터링을 걸어준 이후, data 값을 확인한 결과,

#)EV@!.n3d1\V,P Alright, I'll send the message through image.

와 같은 문장을 확인할 수 있었습니다.

그렇기에, 위에서 설정한 필터링을 걸어둔 상태에서 pcap 파일을 분리한 이후, 추가 분석을 진행하였습니다.

No.	Time	Source	Destination	Protocol	Length	Info
100	124.754002	198.51.100.23	192.168.118.128	TCP	54	[TCP Keep-Alive] 80 → 28955 [ACK] Seq=0 Ack=3600 Win=8192 Len=0
109	124.755002	192.168.118.128	198.51.100.23	TCP	1254	28955 → 80 [PSH, ACK] Seq=3600 Ack=0 Win=8192 Len=1200
110	124.760002	198.51.100.23	192.168.118.128	TCP	54	[TCP Keep-Alive] 80 → 28955 [ACK] Seq=0 Ack=4800 Win=8192 Len=0
111	124.760003	192.168.118.128	198.51.100.23	TCP	823	28955 → 80 [PSH, ACK] Seq=4800 Ack=0 Win=8192 Len=769
112	124.770003	198.51.100.23	192.168.118.128	TCP	54	[TCP Keep-Alive] 80 → 28955 [ACK] Seq=0 Ack=5569 Win=8192 Len=0
113	124.780003	192.168.118.128	198.51.100.23	TCP	54	28955 → 80 [FIN, ACK] Seq=5569 Ack=0 Win=8192 Len=0
114	124.790003	198.51.100.23	192.168.118.128	TCP	54	[TCP Keep-Alive] 80 → 28955 [ACK] Seq=0 Ack=5570 Win=8192 Len=0
115	135.880003	192.168.118.128	198.51.100.23	TCP	54	47829 → 80 [SYN] Seq=0 Win=8192 Len=0
116	135.810003	198.51.100.23	192.168.118.128	TCP	54	80 → 47829 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0
117	135.820003	192.168.118.128	198.51.100.23	TCP	54	47829 → 80 [ACK] Seq=1 Ack=1 Win=8192 Len=0
118	135.830003	198.51.100.23	192.168.118.128	TCP	1254	80 → 47829 [PSH, ACK] Seq=0 Ack=0 Win=8192 Len=1200
119	135.840003	192.168.118.128	198.51.100.23	TCP	54	[TCP Keep-Alive] [TCP ACKed unseen segment] 47829 → 80 [ACK] Seq=0 Ack=1200 Win=8192 Len=0
120	135.842003	198.51.100.23	192.168.118.128	TCP	1254	80 → 47829 [PSH, ACK] Seq=1200 Ack=0 Win=8192 Len=1200
121	135.852003	192.168.118.128	198.51.100.23	TCP	54	[TCP Keep-Alive] 47829 → 80 [ACK] Seq=0 Ack=2400 Win=8192 Len=0
122	135.854003	198.51.100.23	192.168.118.128	TCP	1254	80 → 47829 [PSH, ACK] Seq=2400 Ack=0 Win=8192 Len=1200
123	135.864003	192.168.118.128	198.51.100.23	TCP	54	[TCP Keep-Alive] 47829 → 80 [ACK] Seq=0 Ack=3600 Win=8192 Len=0
124	135.866003	198.51.100.23	192.168.118.128	TCP	1254	80 → 47829 [PSH, ACK] Seq=3600 Ack=0 Win=8192 Len=1200
125	135.876003	192.168.118.128	198.51.100.23	TCP	54	[TCP Keep-Alive] 47829 → 80 [ACK] Seq=0 Ack=5569 Win=8192 Len=0
126	135.878003	198.51.100.23	192.168.118.128	TCP	858	80 → 47829 [PSH, ACK] Seq=4800 Ack=0 Win=8192 Len=796
127	135.880003	192.168.118.128	198.51.100.23	TCP	54	[TCP Keep-Alive] 47829 → 80 [ACK] Seq=0 Ack=5596 Win=8192 Len=0
128	135.890003	198.51.100.23	192.168.118.128	TCP	54	80 → 47829 [FIN, ACK] Seq=5596 Ack=0 Win=8192 Len=0
129	135.900003	192.168.118.128	198.51.100.23	TCP	54	[TCP keep-alive] 47829 → 80 [ACK] Seq=0 Ack=5597 Win=8192 Len=0
130	146.910003	198.51.100.23	192.168.118.128	TCP	54	60000 → 54321 [FIN, ACK] Seq=164 Ack=156 Win=8192 Len=0
131	146.920003	192.168.118.128	198.51.100.23	TCP	54	54321 → 60000 [ACK] Seq=156 Ack=165 Win=8192 Len=0

Internet Protocol Version 4, Src: 198.51.100.23, Dst: 192.168.118.128

Transmission Control Protocol, Src Port: 80, Dst Port: 47829, Seq: 0, Ack: 0, Len: 1200

Source Port: 80

Destination Port: 47829

[Stream Index: 8]

[Conversation completeness: Complete, WITH_DATA (31)]

[TCP Segment (raw): 31879]

Sequence Number: 0 (relative sequence number)

[Next Sequence Number: 1200 (relative sequence number)]

Acknowledgment Number: 0 (relative ack number)

Acknowledgment number (raw): 7895

0101 ... = Header Length: 20 bytes (5)

Flags: 0010 [PSH, ACK]

Window: 8192

[Calculated window size: 8192]

[Window size scaling factor: -2 (no window scaling used)]

Checksum: 0x0209 [Unverified]

[Checksum Status: Unverified]

Urgent Pointer: 0

[Timestamps]

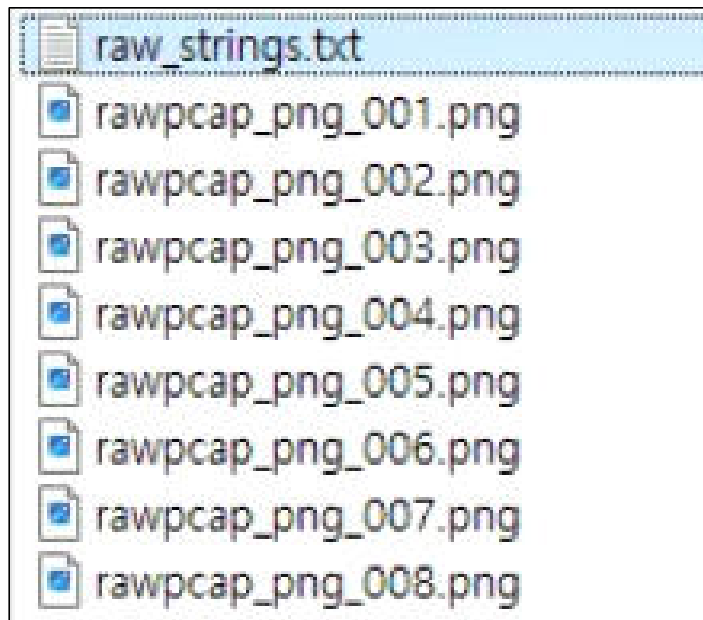
[SEQ/ACK analysis]

[Client Contiguous Streams: 1]

[Server Contiguous Streams: 1]

TCP payload (1200 bytes)

추가 분석 결과, 해당 사진과 같이 png 파일들이 존재하는 것을 확인할 수 있었으며, 패킷 파일 내에서 파일 및 strings 내용을 추출하였습니다.



추출 결과, 총 8개의 이미지 파일과 raw_strings.txt 내용을 확보할 수 있었습니다.

먼저, raw_strings.txt 파일 내 담긴 대화 내용은 아래와 같습니다.

From now on, let's hide our messages inside images.

Inside images? How would anyone know to look for them?

I'll put a four-digit length at the front. Just read up to that.

Got it. How can I notice that an image has a message?

I'll invert the red channel so you can notice.

Alright, I'll send the message through image.

위 대화 내용을 확인한 이후, 내릴 수 있는 결론은 아래와 같습니다.

- 송신자가 메시지를 숨긴 이미지의 'Red(빨간색) 채널' 값을 반전시켜서 보냅니다.
- 송신자는 숨겨진 메시지 데이터의 맨 앞에, 그 메시지의 총 길이를 나타내는 4자리 숫자를 함께 숨깁니다.
- 메시지를 추출하기 위해서는, 해당 4자리 숫자를 먼저 읽은 후, 뒤따라올 메시지의 길이를 파악한 뒤, 해당 길이만큼의 데이터를 읽어 메시지를 복원합니다.

위 대화 내용을 기반으로, 송신자가 스테가노그래피 기법을 활용하여, 이미지 파일 내 정보를 숨겼다는 것을 파악할 수 있었으며,

해당 내용을 확인하기 위해 stylesuxx.github.io에서 제공하는 스테가노그래피 도구를 활용하게 되었습니다. (<https://stylesuxx.github.io/steganography/>)



위에서 언급한 스테가노그래피 도구를 통해, 제품명(HelioKey) 및 날짜(2025-12-25)를 확보한 이후, 플래그를 아래와 같이 조합하고 제출하였습니다.

최종 플래그는 다음과 같습니다.

whitehat2025{HelioKey_2025-12-25}